Version Control

16/09/2026 Aryan Singh

Introduction to Version Control

Version control is a system which keeps track and helps to manage all the changes made to files over time. It is especially useful when working on software projects, where programmers need to continuously modify their codes.

One of the most widely used version control systems today is Git. It allows developers to save different versions of their projects, compare changes, and a lot more.

Version Control Systems work using a few core concepts that help teams manage code changes and collaborate efficiently.

- ➢ Repository: A central location that stores all project files along with their complete change history and data like author and commit message.
- ➢ Revision: A specific saved version of a file or project, identified using a unique ID such as a hash or number.
- ➢ Branch: A separate copy of the codebase used to develop features or fix bugs without affecting the main code.
- ➢ Merging: The process of combining changes from one branch into another
- ➢ Commit: A snapshot of changes made to the codebase at a specific time, used to track and manage project history.

In short, version control is like the history of a project. It tells us what was changed, who changed it, when the change was made and combine the changes, making the work way easier when working in a team.

Importance of Version Control

Version control is important due to many reasons like :

a) It keeps history of changes made

Version control records the changes made to a project. This is very useful as if a programmer changes a section of code today and the program stops working, they can check the previous version, identify what was changed and figure out the error. It also is very helpful, if a developer

accidentally removes something, as it can be recovered through the previous versions. And in larger projects since it records information about changes made, including the people who made them. This makes it easier for a team to understand the development history of a project.

b) Makes Teamwork Easier

In a group project, several people may need to work on the same code. Hence, version control allows everyone to create separate branches for their work, make their own changes and later combine them into the main project. This is very very useful for large software projects where many developers are working simultaneously.

c) Supports Experimentation

Developers can create a separate branch and experiment with a new feature without immediately affecting the main version of the project. If the experiment works, it can later be merged into the main branch. Therefore, version control is useful even for safely managing changes, trying new features and improving the project, without affecting the live version instantaneously.

Types of Version Control

Version control systems can broadly be divided into centralized and distributed systems. Local version control can also be considered an earlier/basic approach to managing versions.

Local Version Control

In a local version control system, different versions of files are maintained on the same computer. This approach can work for a single person and a small project, but it has limitations as there is no convenient system for collaboration, and losing the computer or its storage could result in losing the project history. Local version control therefore provides basic version tracking but is not ideal for larger team-based projects.

Centralized Version Control

In a Centralized Version Control System (CVCS), a central server stores the project's files and version history. Developers connect to this central repository to obtain the project and submit their changes.

For example, Subversion (SVN) and Perforce.

The main advantage of this approach is that everyone works with a common central repository. However, the central server becomes an important part of the workflow. If it is unavailable, some version control operations and collaboration may be affected.

Distributed Version Control

In a Distributed Version Control System (DVCS), each developer has a complete copy of the repository, including its history.

Git and Mercurial are examples of distributed version control systems.

This approach provides more flexibility because many operations can be performed locally without continuously depending on a central server. Developers can make commits, examine history, and work with branches on their own computers. Changes can later be shared with other repositories.

Apart from these, there are some specialised models of Version Control like:

Lock Based Version Control

A lock-based version control system uses file locking to manage concurrent access to files and resources. File locking prevents two or more users from making conflicting changes to the same file or resource in large binary or proprietary assets, which comprises of complex data, not represented as plain text, in terms of rovers are 3D models of rover, high resolution maps, sensor logs, etc.

Semantic / Lineage-Based Version Control

It helps to train an AI model, for example in terms of rovers if we need to train a rover on 50 gigabytes of terrain photos so the rover can recognize rocks.
These massive datasets, video logs, or trained model weights will be stored on a dedicated storage server or cloud bucket like AWS S3.
The semantic versioning tool (like DVC) creates a tiny, human-readable text file for example, “terrain_dataset.dvc” containing a unique cryptographic hash of that massive dataset.
When a teammate goes to an older code branch, the pointer changes, telling the system to automatically download the exact version of the massive dataset that matched that specific historical moment of the code.

Important Git Commands

Few of the common commands are:

1. git init

This command creates a new Git repository in a project directory. It is generally used when starting version control for a new local project.

2. git clone

This command creates a local copy of an existing remote repository. It is useful when a developer wants to start working on a project that already exists online. The clone contains the project's files, history, and branches.

3. git status

This command shows the current state of the working directory. It helps identify files that are untracked, modified, or staged.

4. git add

This command stages changes so that they can be included in the next commit. The staging step allows the developer to choose which changes should be included in the next commit.

5. git commit

A commit records staged changes in the project's history. The message in the commit describes what was changed and makes the project's history easier to understand.

6. git log

This command displays the commit history of the repository. It can be used to examine previous commits and understand how the project has developed.

7. git branch

This command is used to view or create branches. Branches are useful when developers want to work on a feature separately from the main development line.

8. git switch

This command is used to switch from one branch to another. It allows a developer to move between different lines of development.

9. git merge

This command combines the changes from one branch with another branch. For example, after completing and testing a feature on a separate branch, its changes can be merged into the main branch.

10. git pull

This command obtains changes from a remote repository and integrates them into the current local branch. It is particularly useful in team projects where other members may have pushed new changes to the shared repository.

11. git push

This command sends local commits to a remote repository. It allows other members of the team to access the changes that have been committed locally.

6.12 git remote

This command is used to manage connections to remote repositories. It can be used to view the remote repositories connected to the local project.